STUDENT MANAGEMENT SYSTEM

Object Oriented Software Development Lab [CSS_2213]
Mini Project Report submitted to
Manipal Academy of Higher Education
in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Information Technology

Submitted by

Anchit Goel

245811180

Under the guidance of

Ms. Divya Shree
Assistant Professor – On Contract
School of Computer Engineering
Manipal Institute of Technology



MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)

MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)
SCHOOL OF COMPUTER ENGINEERING

Bengaluru
12-04-2026

CERTIFICATE

This is to certify that the Mini project titled Student Management System is a record of the Bonafide work done by **Anchit Goel (*Reg. No. 245811180*)** submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech.) in **INFORMATION TECHNOLOGY** of Manipal Institute of Technology, Bengaluru, Karnataka, (A Constituent Institute of Manipal Academy of Higher Education), during the academic year 2025-2026.

Ms. Divya Shree
Assistant Professor – On Contract
SoCE, MIT Bengaluru

Dr. Preethi
Program Coordinator, Assistant Professor
SoCE, MIT Bengaluru

ABSTRACT

The Student Course Registration System is a desktop-based application developed using JavaFX to demonstrate core software engineering principles in a simplified and practical manner. The system is designed using the Model-View-Controller (MVC) architecture and employs FXML for creating an interactive graphical user interface. The application enables users to add students and courses, register students to courses, view records dynamically, and delete entries as needed.

Unlike traditional systems that rely on complex database integrations, this application uses in-memory data storage through ArrayList and ObservableList, making it lightweight, portable, and easy to deploy. The project is particularly suitable for academic demonstrations and beginner-level understanding of JavaFX-based application development.

INTRODUCTION

Efficient management of student-course registrations is essential for academic institutions to ensure smooth coordination between students and course offerings. However, many existing systems depend on complex backend infrastructures and database management systems, which can be difficult to implement and maintain, especially for small-scale or educational purposes.

This project presents a simplified solution in the form of a desktop-based application developed using JavaFX. The system provides a user-friendly graphical interface that allows users to perform essential operations such as adding students, managing courses, and registering students in courses. By leveraging JavaFX and the MVC design pattern, the application ensures a clear separation between the user interface and application logic, enhancing both readability and maintainability.

The primary objective of this project is to provide a practical learning platform for beginner Java developers while also serving as a demonstration tool in academic environments. It focuses on fundamental concepts such as event handling, data binding, and modular design without introducing unnecessary complexity.

SYSTEM REQUIREMENTS

The system is designed to operate in a standard computing environment with minimal hardware and software requirements. It can be executed on any personal computer or laptop with basic processing capabilities, without the need for specialized hardware components.

The application is compatible with multiple operating systems, including macOS, Windows, and Linux. It requires Java Development Kit (JDK) version 17 or higher, with the project developed and tested using JDK 23. Additionally, the JavaFX Software Development Kit (SDK) is required to support the graphical user interface components, and a compatible OpenJFX distribution must be configured properly.

The project can be developed and executed using Visual Studio Code or any Java-supported integrated development environment. Optional tools such as Git may be used for version control and collaboration. Since the application uses in-memory storage and does not rely on

external databases, it has minimal memory and storage requirements, making it efficient and easy to run in standard development environments.

SYSTEM DESIGN

a) Modules

The system is designed using a modular approach following the Model-View-Controller (MVC) architecture, which ensures separation of concerns and enhances maintainability. The Model component is responsible for representing the core data entities of the application, namely Student, Course, and Registration. Each of these classes encapsulates relevant attributes and provides methods for accessing and manipulating data. The Student and Course classes maintain unique identifiers and names, while the Registration class establishes a relationship between a student and a course, ensuring that each registration instance is uniquely identifiable.

The View component is implemented using FXML, which defines the graphical user interface of the application. It provides a structured and user-friendly layout using JavaFX elements such as TableView, ComboBox, Button, and layout containers like VBox and GridPane. This separation of UI design from application logic enables easier modifications and better readability.

The Controller component acts as the intermediary between the Model and the View. It handles user interactions, processes input data, performs validation, and updates the UI accordingly. The controller maintains in-memory data structures, such as ArrayList and ObservableList, to store and manage application data dynamically. It ensures synchronization between the data model and the user interface, enabling real-time updates.

b) Flow

The overall system flow begins with user interaction through the graphical interface. When a user inputs data, such as adding a student or a course, the controller captures this input and validates it to ensure correctness. Upon successful validation, corresponding model objects are created and stored in in-memory collections. These collections are then wrapped into ObservableLists, which automatically update the UI components like TableView and ComboBox.

For the registration process, the user selects a student and a course from the available options. The controller verifies that both selections are valid and checks for duplicate registrations to maintain data integrity. If the registration is unique, it is added to the system and immediately reflected in the UI. Similarly, when a user deletes a student or course, the controller removes the associated data and updates all dependent views to maintain consistency across the application.

IMPLEMENTATION

The implementation of the system is carried out using Java and JavaFX, adhering to object-oriented programming principles. The project is structured into separate packages to maintain

clarity and modularity. The model classes are implemented with JavaFX properties to enable seamless data binding with UI components. Unique identifiers for students and courses are generated using simple static counters, ensuring that each entity can be distinctly referenced.

The controller is responsible for handling all application logic, including adding new records, performing validations, managing registrations, and updating the UI. It uses ObservableList to maintain synchronization between the underlying data and the graphical components. Event handling is implemented using JavaFX annotations, allowing user actions such as button clicks to trigger specific methods.

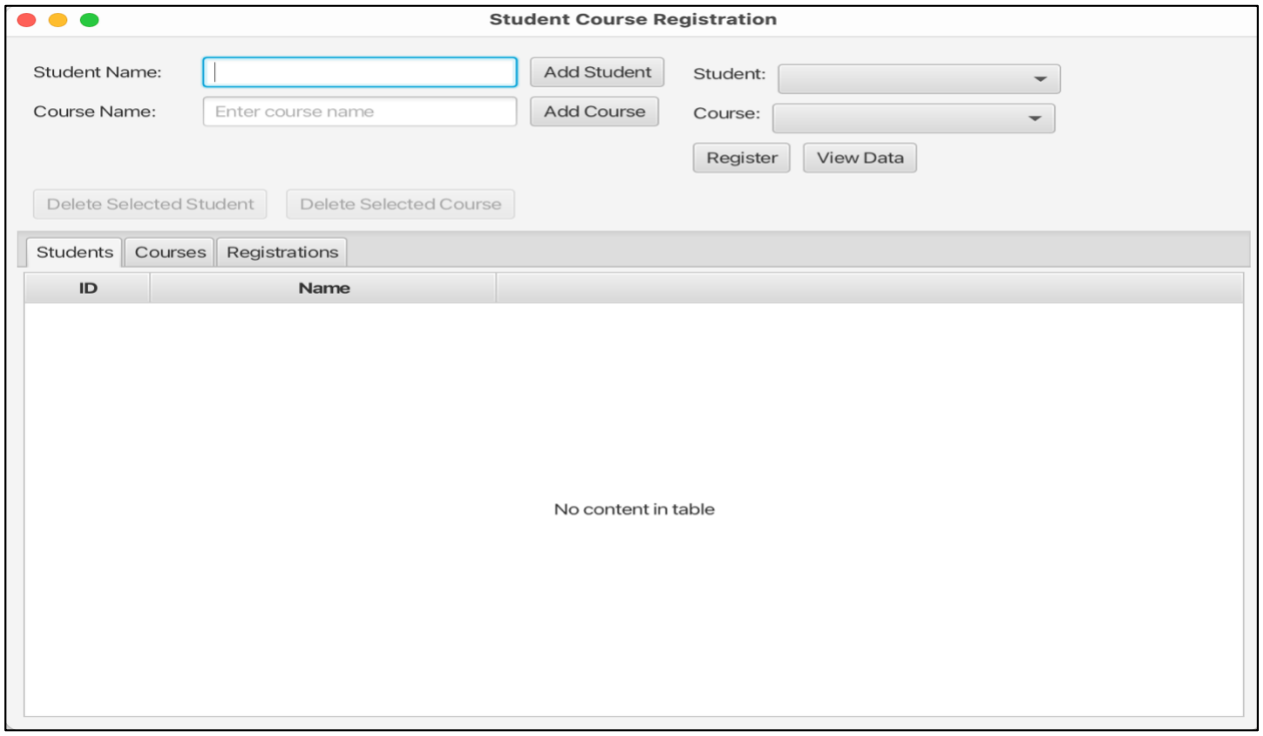
The user interface is developed using FXML, which allows for a clean separation of design and logic. Scene Builder compatibility ensures that the layout can be visually designed and modified without affecting the underlying code. The application is executed using the JavaFX runtime environment, with proper configuration of module paths and required libraries.

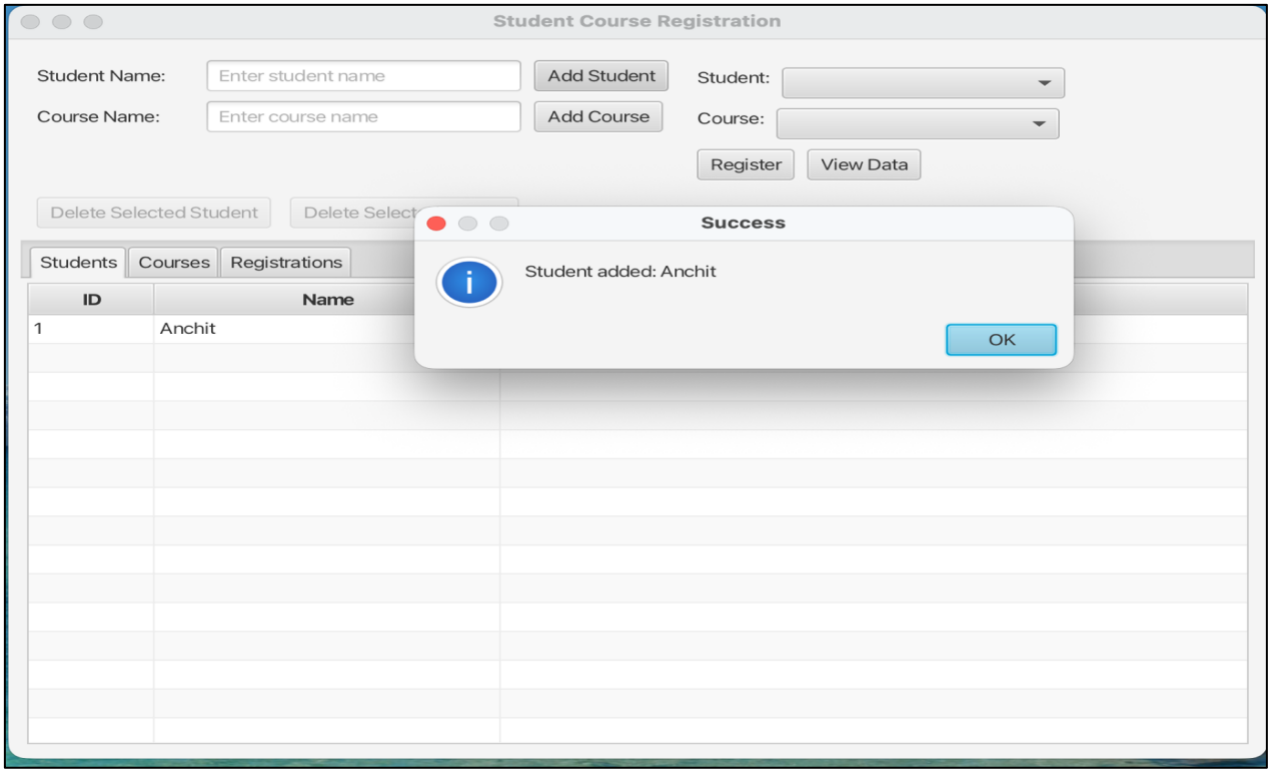
FEATURES OF THE APPLICATION

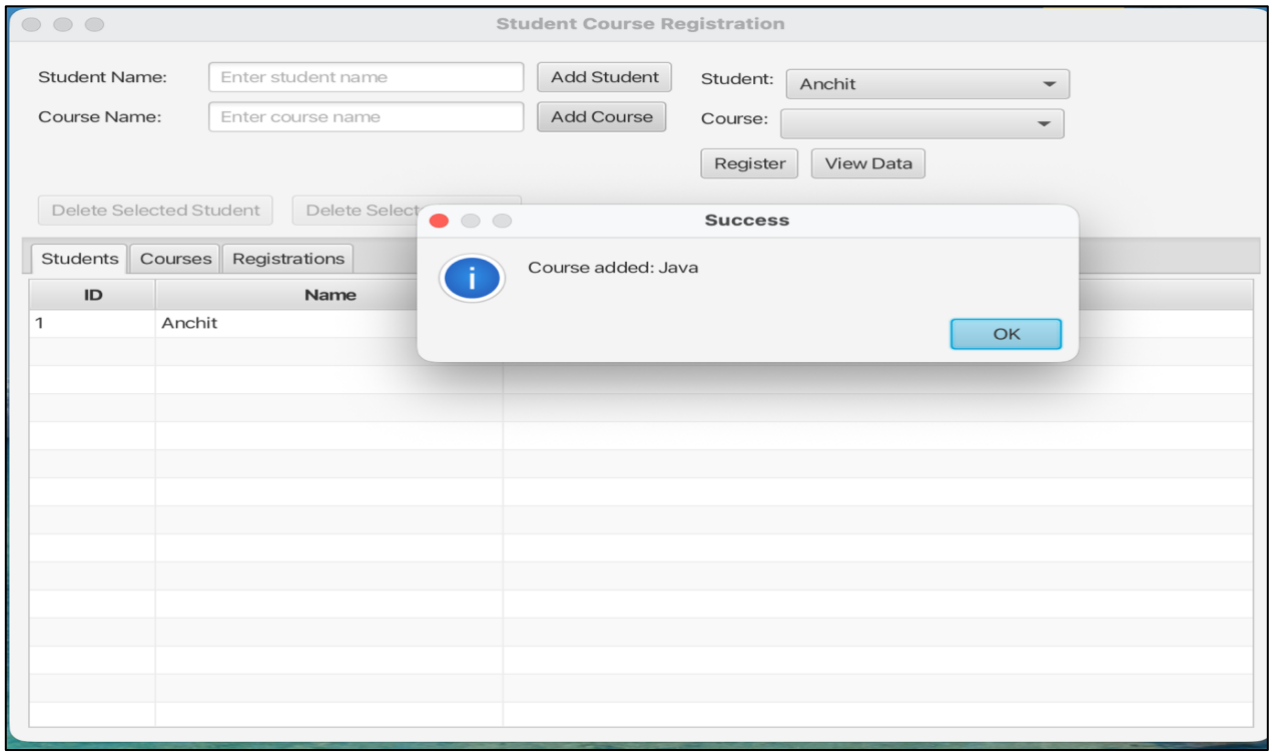
The application provides a comprehensive yet simple set of features to manage student-course registrations effectively. It allows users to add new students and courses with automatically generated identifiers, ensuring uniqueness and ease of management. The system enables users to register students to courses through an intuitive selection mechanism using dropdown menus. It prevents duplicate registrations, thereby maintaining data consistency and integrity.

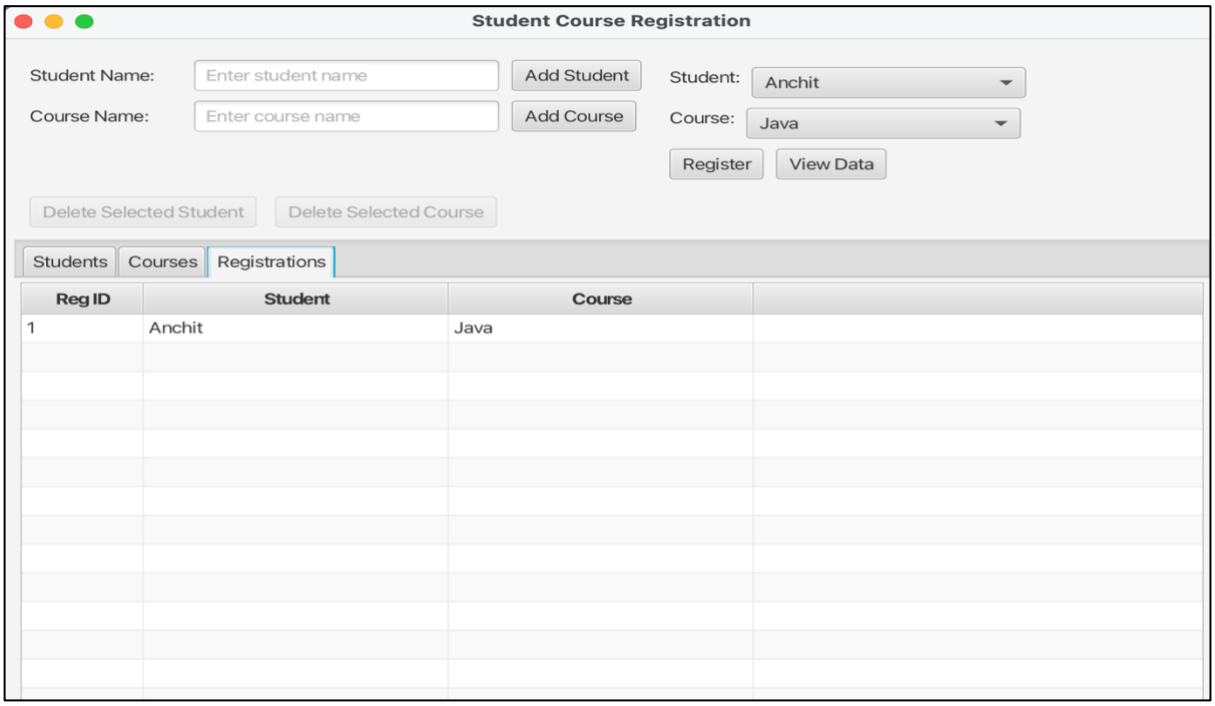
The application also includes functionality to view all stored data in a structured format using TableView components, which dynamically update as changes are made. Additionally, users can delete existing students or courses, and the system ensures that any related registrations are also removed to avoid inconsistencies. The application provides user feedback through alert dialogs, enhancing usability and interaction.

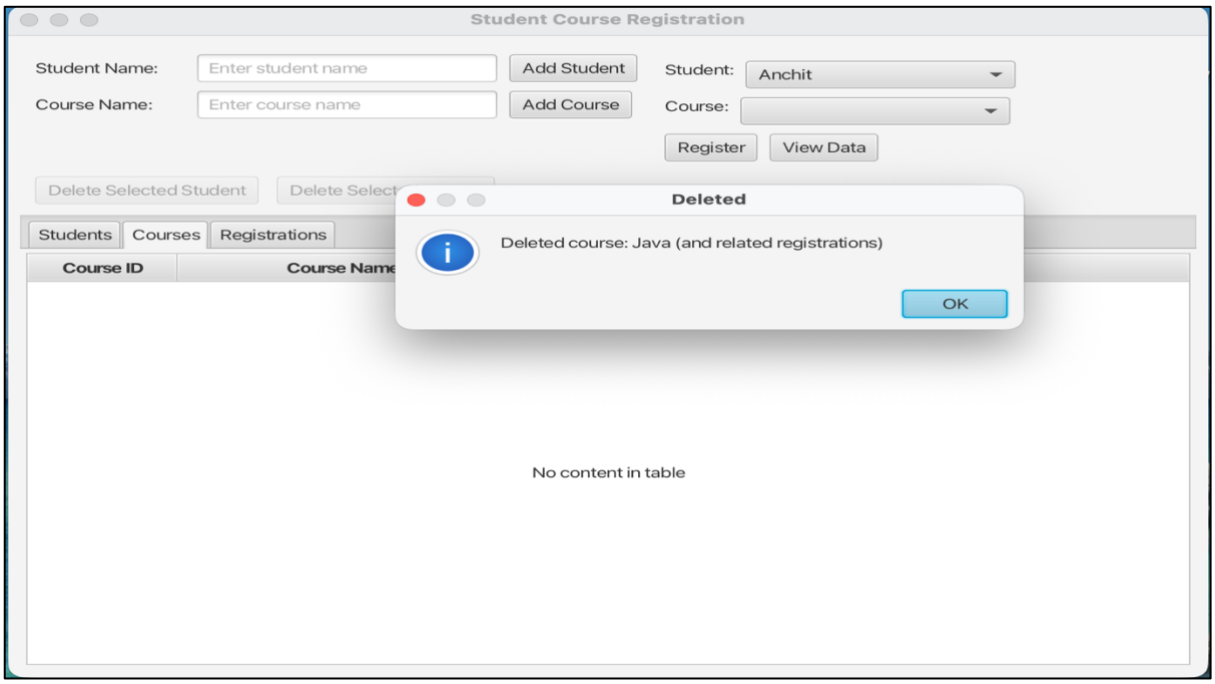
SCREENSHOTS OF PROJECT

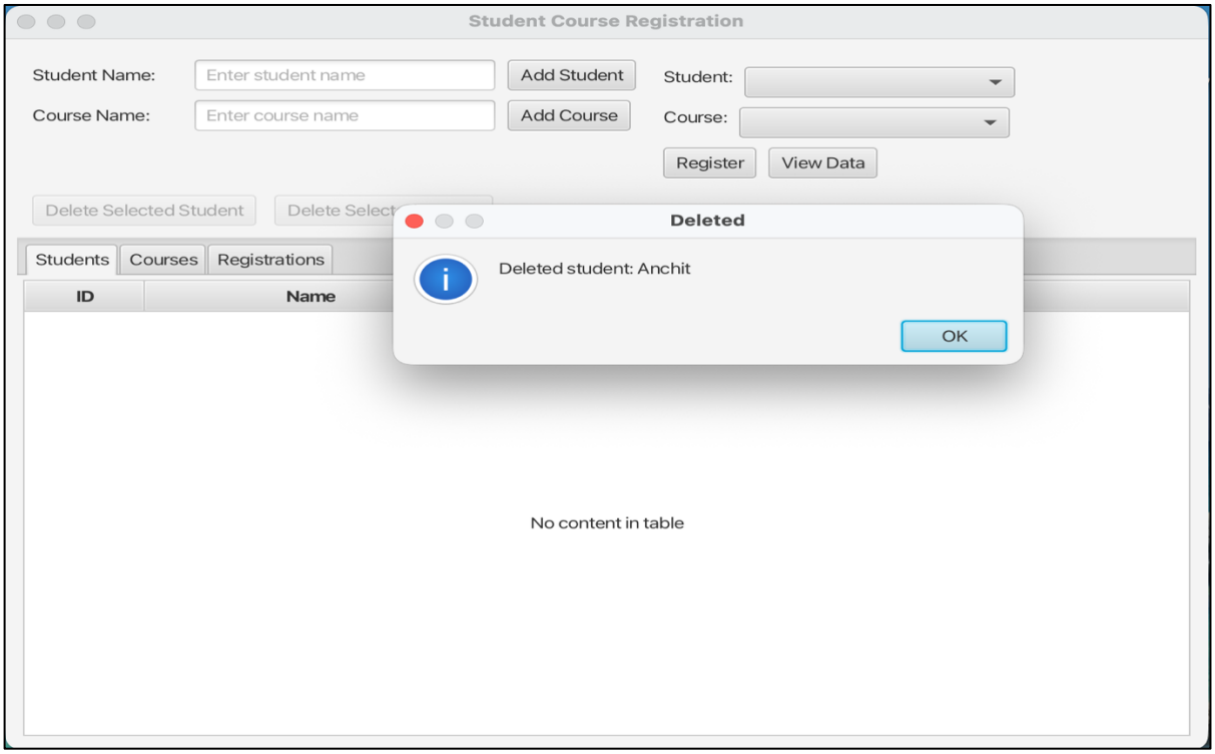



Student Course Registration
Student Name:
Enter student name
Add Student
Student:
Course Name:
Enter course name
Add Course
Course:
Register
View Data
Delete Selected Student
Success
Student added: Anchit
OK
Students
Courses
Registrations
ID
Name
1
Anchit


Student Course Registration
Student Name:
Enter student name
Add Student
Student:
Anchit
Course Name:
Enter course name
Add Course
Course:
Register
View Data
Delete Selected Student
Success
Course added: Java
OK
Students
Courses
Registrations
ID
Name
1
Anchit


Student Course Registration
Student Name:
Enter student name
Add Student
Student:
Anchit
Course Name:
Enter course name
Add Course
Course:
Java
Register
View Data
Delete Selected Student
Delete Selected Course
Students
Courses
Registrations
Reg ID
Student
Course
1
Anchit
Java


Student Course Registration
Student Name:
Enter student name
Add Student
Student:
Anchit
Course Name:
Enter course name
Add Course
Course:
Register
View Data
Delete Selected Student
Students
Courses
Registrations
Course ID
Deleted
Deleted course: Java (and related registrations)
OK
No content in table



CONCLUSION

In conclusion, the Student Course Registration System successfully demonstrates the development of a desktop-based application using JavaFX and MVC architecture. The project highlights key concepts such as modular design, event-driven programming, data binding, and user interface development using FXML. By utilizing in-memory data storage, the system remains lightweight and easy to deploy, making it highly suitable for academic and demonstration purposes.

The application achieves its objective of providing a simple yet functional solution for managing student-course registrations without the need for complex backend infrastructure. It serves as a strong foundation for further enhancements, such as database integration, advanced user interfaces, and additional functionalities, thereby offering significant scope for future development.
